

claude-windows / c6028e72-ebfd-4218-89a8-c4d7dd23be96

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c6028e72-ebfd-4218-89a8-c4d7dd23be96\subagents\workflows\wf_d251ad6c-472\agent-acc0de4f0237d2696.jsonl`  
- SHA-256: `53d6fffdb68977994e08517ea2e3471f971e6c474d8297d1f6252d23e7c3a8c5`  
- Source modified: `2026-06-08T08:45:31+00:00`  
- Imported at: `2026-07-05T16:48:23+00:00`  
- project: `wf_d251ad6c-472`  
- session_id: `c6028e72-ebfd-4218-89a8-c4d7dd23be96`
```

Transcript

1. user (2026-06-08T08:43:41.062Z)

LENS = CANONICAL-HUB ARCHITECT. Propose positioning the collector as the SINGLE canonical ingestion/validation/registration/export HUB for ALL annotation sources (manual self-rating via the VLC annotator, vendor CVAT, vendor CSV review). Design the canonical schema + adapters so every source normalizes into one rally record, and the indexer only ever consumes the collector's export. Emphasize clean adapters, one source of truth, extensibility to new sports/sources.

Ground every recommendation in these subsystem FINDINGS (cite files). Do not invent files or fields that the findings do not support.

FINDINGS:

```
[  
  {  
    "subsystem": "Sports Data Collector - Local/Manual Ingestion CLI",  
    "summary": "The sports-data-collector provides a manual ingestion pipeline (`import-local` command) that accepts local video files paired with JSON or CSV annotation files in a sport-specific rally/point/delivery format. It parses annotations, validates temporal consistency (no overlaps, ordered progression), registers metadata + segments in SQLite, and exports curated data as indexer-ready eval CSVs containing only (start, end) decimal-second intervals per video.",  
    "key_facts": [  
      "CLI entry: src/cli/curate.py:844-961 (main function with subparser setup); import-local command registered line 854-863",  
      "import_local() method: src/cli/curate.py:52-155; accepts --sport (required, lowercase), --video-path (required), --annotations-path (required), --match-name (optional, defaults to video basename), --video-id (optional, defaults to lowercased match_name), --license (default='Proprietary'), --fps (default=30.0), --resolution (default='1080p')",  
      "_parse_local_annotations() method: src/cli/curate.py:156-302; accepts .json or .csv files; normalizes CSV headers to lowercase; raises ValueError if unsupported extension",  
      "Badminton schema: src/core/schemas/badminton.py:4-22; required fields: video_id, rally_number (int), start_time (float), end_time (float); optional: score_before, score_after, shots_count (int), ending_reason, last_shot_outcome; validates end_time > start_time",  
      "CSV parsing: src/cli/curate.py:232-298; uses csv.DictReader with normalized lowercase headers; safe_int() with idx+1 default for rally_number; safe_float_required() for start_time, end_time (raises ValueError if missing/empty); gracefully degrades optional fields to None",  
      "JSON parsing: src/cli/curate.py:161-231; expects list of objects; uses identical safe_int/safe_float logic; treats missing required fields (start_time, end_time) as ValueError",  
      "Validation pipeline: src/cli/curate.py:88-106 calls validator.validate_badminton_rallies() at src/core/validator.py:29-57; checks: no zero/negative duration (start_time < end_time), no temporal overlaps between sequential rallies (sorted by rally_number), chronological progression; returns (bool, List[str]) errors",  
      "DB insertion: src/cli/curate.py:123-136 inserts VideoMetadata (status=CURATED for manual imports, bypassing staging) + rallies; db.insert_badminton_rallies() at src/core/db.py:210-237 performs UPSERT (DELETE then INSERT per video_id)",  
    ]  
  }  
]
```

```

    "Export format: export_eval_set() at src/cli/curate.py:440-536; emits per-video CSV with
header [start,end] + decimal-second tuples (queried via get_segment_intervals() at
src/core/db.py:527-546); manifest.json pairs CSVs with video metadata (local_path, video_url,
fps, resolution, etc.)",
    "License handling: LicenseType enum at src/core/schemas/base.py:12-22 (MIT, Apache-2.0,
BSD, CC0, CC-BY, CC-BY-NC, CC-BY-NC-SA, Public-Domain, Proprietary, Unknown);
validator.check_license_commercial() at src/core/validator.py:24-27 checks against whitelist
(config default: MIT, Apache-2.0, BSD, CC0, CC-BY, Public-Domain, Proprietary)",
    "CurationStatus enum: src/core/schemas/base.py:24-27 (STAGING, CURATED, REJECTED); manual
imports set status=CURATED directly (line 117)",
    "Rally-annotator CSV format compatibility: docs/CSV_FORMAT.md specifies columns
(rally_number, start_time, end_time, ending_reason, sport); decimal seconds, 1-based rally
numbers; sports-data-collector import-local reads this directly (compatible via DictReader
header matching)"
],
"contracts": [
    "import-local CLI flags: --sport (required:
badminton|tennis|cricket|pickleball|tabletennis), --video-path (required: file path),
--annotations-path (required: .json or .csv), --match-name (str, optional), --video-id (str,
optional), --license (str, default='Proprietary'), --fps (float, default=30.0), --resolution
(str, default='1080p')",
    "CSV input columns (required): rally_number (int or auto-indexed), start_time (float,
decimal seconds), end_time (float, decimal seconds); optional: score_before, score_after,
shots_count, ending_reason, last_shot_outcome. Case-insensitive header matching via lowercase
normalization.",
    "JSON input structure: list of objects, each with keys: rally_number (int), start_time
(float), end_time (float), plus optional fields matching CSV",
    "Badminton rally object contract: video_id (str), rally_number (int), start_time (float),
end_time (float); start_time must be < end_time; rally_number serves as chronological sort key",
    "DB schema: badminton_rallies table (video_id, rally_number, start_time, end_time,
score_before, score_after, shots_count, ending_reason, last_shot_outcome) with PK=(video_id,
rally_number), FK on videos.video_id with ON DELETE CASCADE",
    "Export format: per-video CSV at <out_dir>/<sport>/<video_id>.csv with header [start,end]
and (start_time, end_time) decimal-second tuples in chronological order; manifest.json with
entries {video_id, sport, csv, local_path, video_url, match_name, fps, resolution, license_type,
is_commercial, status, num_segments}"
],
"gaps_or_mismatches": [
    "Fractional rally_number support: rally-annotator can produce decimal rally_number (e.g.,
2.5 for resegmented rallies); sports-data-collector expects int, converts via safe_int() which
truncates (2.5 -> 2), causing loss of distinction between fractional variants; no explicit error
on truncation",
    "Partial coverage handling: rally-annotator CSV may have gaps (e.g., rally 1,2,4 missing
3) or float numbers (1.1, 1.2); validator checks only temporal non-overlap and chronological
order via rally_number sort, not contiguity or valid numbering schemes",
    "Missing optional fields: sport column in rally-annotator CSV is self-documenting but
ignored by import-local; no validation that CSV sport matches --sport flag",
    "Score/outcome fields: rally-annotator tracks ending_reason enum
(winner|forced_error|unforced_error|service_fault|let|other); maps exactly to
BadmintonRally.ending_reason (optional); no score_before/score_after in rally-annotator, must be
None in import",
    "Null/empty handling: safe_float_required() treats empty string as ValueError; optional
int fields default to None (not 0) via safe_int(..., None) if missing; no distinction between
truly-missing and user-entered null",
    "Time format flexibility: parser supports only decimal seconds (float); does NOT support
mm:ss or HH:MM:SS time strings (unlike some consuming tools), only raw float values"
],
"files": [
    "C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py",
    "C:/Users/avidu/Projects/sports-data-collector/src/cli/ingest.py",
    "C:/Users/avidu/Projects/sports-data-collector/src/core/db.py",
    "C:/Users/avidu/Projects/sports-data-collector/src/core/parsing.py",
    "C:/Users/avidu/Projects/sports-data-collector/src/core/validator.py",
    "C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/base.py",
    "C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/badminton.py",

```

```

"C:/Users/avidu/Projects/rally-annotator/docs/CSV_FORMAT.md"
]
},
{
  "subsystem": "Canonical Rally Schema & CSV I/O (sports-data-collector)",
  "summary": "The sports-data-collector defines a canonical rally record schema
(BadmintonRally/PickleballRally/TableTennisRally) with required fields video_id, rally_number,
start_time, end_time, and optional fields score_before/after, shots_count, ending_reason,
last_shot_outcome. Validation enforces non-negative start times, strictly positive durations,
chronological ordering, and no temporal overlaps. The canonical CSV (ROORA_BRIEF format)
includes 11 columns: rally_number, start_mmss, end_mmss, start_time, end_time, shots_count,
ending_reason, last_shot_outcome, score_before, score_after, notes. CVAT XML is converted to
this canonical format; import-local CSV ingest and export to eval sets both read/write canonical
columns only, dropping all other fields.",
  "key_facts": [
    "BadmintonRally schema: C:/Users/avidu/Projects/sports-data-
collector/src/core/schemas/badminton.py:4-22. Required: video_id (str), rally_number (int),
start_time (float), end_time (float). Optional: score_before/score_after (str), shots_count
(int), ending_reason (str), last_shot_outcome (str).",
    "Validation: C:/Users/avidu/Projects/sports-data-collector/src/core/validator.py:29-57
(validate_badminton_rallies). Rules: start_time >= 0, end_time > start_time
(Pydantic+validator), no temporal overlap between consecutive rallies, sorted by rally_number.",
    "Canonical CSV read: C:/Users/avidu/Projects/sports-data-
collector/src/parsers/cvat/rally_cvat.py:424-460 (load_canonical_csv). Reads decimal
start_time/end_time or mm:ss start_mmss/end_mmss; normalizes headers to lowercase; skips rows
missing rally_number or usable times; frame_min/frame_max set to -1 (CSV has no frame info).",
    "Canonical CSV write: C:/Users/avidu/Projects/sports-data-
collector/src/parsers/cvat/rally_cvat.py:356-372 (write_canonical_csv). 11 columns:
rally_number, start_mmss, end_mmss, start_time (%.3f), end_time (%.3f), shots_count,
ending_reason, last_shot_outcome, score_before, score_after, notes. Empty fields written as
blank strings.",
    "Canonical CSV columns (ROORA_BRIEF): C:/Users/avidu/Projects/sports-data-
collector/docs/annotation/ROORA_BRIEF.md:18-30. Enum ending_reason: 'winner' | 'forced_error' |
'unforced_error' | 'service_fault' | 'let' | 'other'. Enum last_shot_outcome: 'in' | 'net' |
'out' | 'n_a' (n_a for service_fault/let).",
    "Import-local CSV parse: C:/Users/avidu/Projects/sports-data-
collector/src/cli/curate.py:232-250. Reads CSV with DictReader, normalizes headers lowercase,
uses safe_int/safe_float for optional fields (defaults to None), required fields
(start_time/end_time) raise ValueError if missing. No sport/status/maturity/fractional columns
read; only 9 fields map to BadmintonRally.",
    "Export eval set: C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py:440-536.
Writes per-video CSV with 2 columns: start, end (decimal seconds only). No rally_number,
ending_reason, last_shot_outcome, shots_count, or scores in export; uses get_segment_intervals
(start_time, end_time only from database).",
    "CVAT conversion: C:/Users/avidu/Projects/sports-data-
collector/src/parsers/cvat/rally_cvat.py:325-353 (convert). Time recomputed from
frame_min/frame_max: start_time = frame_min / fps, end_time = frame_max / fps (vendor
start_time/end_time attributes discarded). Issues reported (not auto-fixed): zero_duration,
ending_reason_vocab, outcome_vocab, shot_density, frame_out_of_range, overlap,
possible_missed_rally.",
    "Issue detection allowed vocabularies: C:/Users/avidu/Projects/sports-data-
collector/src/parsers/cvat/rally_cvat.py:50-61. DEFAULT_ALLOWED_REASONS: {'winner',
'forced_error', 'unforced_error', 'service_fault', 'let', 'other'}. DEFAULT_ALLOWED_OUTCOMES:
{'in', 'net', 'out', 'n_a'}.",
    "Database schema: C:/Users/avidu/Projects/sports-data-collector/src/core/db.py:74-88.
badminton_rallies table: video_id (TEXT), rally_number (INTEGER), start_time (REAL), end_time
(REAL), score_before (TEXT), score_after (TEXT), shots_count (INTEGER), ending_reason (TEXT),
last_shot_outcome (TEXT). PK: (video_id, rally_number).",
    "Canonical column set on read (CSV / CVAT): rally_number, start_time (or start_mmss),
end_time (or end_mmss), shots_count, ending_reason, last_shot_outcome, score_before,
score_after, notes. Headers normalized to lowercase; missing optional fields degrade to None.",
    "Canonical column set on write (CSV export): ONLY start, end (decimal seconds). Rally
detail, ending_reason, shots_count, outcomes, scores, and video metadata NOT exported. Separate
manifest.json carries video_id, sport, num_segments, local_path, fps, resolution, license,
status."
  ]
}

```

```

"Canonical CSV in curate.py: C:/Users/avidu/Projects/sports-data-
collector/src/cli/curate.py:559 (write_canonical_csv arg). Called by convert-cvat command;
output CSV imports via import-local.",
"Vendor comparison: C:/Users/avidu/Projects/badminton-highlight-
indexer/backend/eval/eval_partial_labels.py (partial labels harness) and
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/calibrate_wasb.py:34-49
(load_golden_gts). Both read CSV with start_time, end_time columns ONLY; case-insensitive header
matching; no requirement for rally_number, ending_reason, shots_count, scores, or outcome
fields."
],
"contracts": [
    "BadmintonRally (& PickleballRally, TableTennisRally): video_id: str (required),
    rally_number: int (required, ?1, chronological), start_time: float (required, ?0), end_time:
    float (required, >start_time), score_before: str | None (default None), score_after: str | None
    (default None), shots_count: int | None (default None), ending_reason: str | None (default
    None), last_shot_outcome: str | None (default None). Pydantic validation: start_time ? 0;
    end_time > start_time.",
    "Canonical CSV columns (on read from CVAT/import-local): rally_number (int, required),
    start_time (float, required | start_mmss fallback), end_time (float, required | end_mmss
    fallback), shots_count (int | None), ending_reason (str | None, vocab:
    winner/forced_error/unforced_error/service_fault/let/other), last_shot_outcome (str | None,
    vocab: in/net/out/n_a), score_before (str | None, format: 'A-B'), score_after (str | None,
    format: 'A-B'), notes (str | None). Header normalization: all headers lowercased and stripped;
    field matching is lenient (aliases for last_shot_outcome: outcome, last_shot_outcome
    (in/net/out)).",
    "Canonical CSV columns (on write to export CSV): ONLY start (decimal seconds, float), end
    (decimal seconds, float). NO rally_number, ending_reason, shots_count, scores, or outcome.
    Written as 2-column CSV with header row.",
    "Canonical CSV columns (on write to CVAT-converted CSV): 11 columns in order:
    rally_number, start_mmss (format m:ss.mmm), end_mmss, start_time (format %.3f), end_time (format
    %.3f), shots_count (empty string if None), ending_reason (empty if None), last_shot_outcome
    (empty if None), score_before (empty if None), score_after (empty if None), notes (empty if
    None).",
    "Validation (validate_badminton_rallies): Returns (bool, List[str] errors). Checks:
    start_time ? 0, end_time > start_time, sorted chronologically by rally_number, no temporal
    overlap (end[i] <= start[i+1] for consecutive rallies). Blocker errors: timestamp violations,
    overlaps. Both CVAT and CSV paths feed this validator.",
    "CVAT issue detection (detect_issues): Returns List[Issue] with severity ('blocker' |
    'high' | 'medium' | 'low'), code (zero_duration, ending_reason_vocab, outcome_vocab,
    shot_density, frame_out_of_range, overlap, possible_missed_rally), message (human-readable),
    rally_number (optional). Blockers: zero_duration, overlap. NOT: sampling density (step/fps),
    since downstream padding absorbs ~0.5s uncertainty.",
    "Ending reason vocabulary (canonical enum): {'winner', 'forced_error', 'unforced_error',
    'service_fault', 'let', 'other'}. Shared across badminton, tennis, table tennis, pickleball,
    padel. Last-shot-outcome is SEPARATE and orthogonal (winner can have outcome='in'; error can
    have outcome='net'/'out'; service_fault/let have outcome='n_a').",
    "Import-local CSV format: flexible headers (case-insensitive), supports
    start_mmss/end_mmss OR start_time/end_time (mm:ss parsed via formula or direct float).
    rally_number defaults to 1-based row index if missing. shots_count, ending_reason,
    last_shot_outcome, score_before, score_after, notes are all optional (degrade to None). No
    sport/status/maturity/curation-level fields read; those come from CLI args."
],
"gaps_or_mismatches": [
    "ending_reason is PRESENT in canonical schema but NOT in eval CSV export (badminton-
    highlight-indexer loads only start_time/end_time from golden CSVs). Implication: the rally-
    indexer evaluation harness does NOT use ending_reason for scoring; it treats rallies as
    unlabeled intervals.",
    "last_shot_outcome is PRESENT in canonical schema but NOT in eval CSV export (same as
    ending_reason). The indexer calibration does not distinguish rally outcomes.",
    "shots_count is OPTIONAL in canonical schema and in CSV import but NOT captured in export
    CSVs. Database stores it; export drops it. Unclear if downstream uses this field.",
    "Fractional rally numbers (e.g., 1.1, 1.2 for replays/lets) are NOT supported by the
    canonical schema (rally_number is int). ROORA-BRIEF mandates integer 1-based rally_number with
    no gaps. If vendors annotate sub-rallies (e.g., 1st serve fault = 1, 1st let = 1.1, 2nd serve =
    1.2), the integer constraint will reject them or require manual splitting.",

```

"Curation status / maturity metadata (e.g., STAGING vs CURATED, confidence flags, rework needed) are NOT in the canonical CSV. They are stored in the database (videos.status enum: staging/curated/rejected) but do NOT round-trip through CSV export. Annotator/vendor CSVs have no analogous columns.",

"Missing end_time is NOT explicitly handled by the canonical schema. ROORA_BRIEF mandates end_time > start_time always. If a vendor omits end_time or provides end_time <= start_time, load_canonical_csv skips the row silently (no error logged). Import-local raises ValueError if end_time is missing/unparseable.",

"Annotation source / vendor / annotator ID is NOT in the canonical CSV. Video metadata carries match_name and video_id but no annotator/rework-status/version tracking per rally.",

"The canonical CSV is NARROWER on export than on import: export loses ending_reason, last_shot_outcome, shots_count, and scores. A CSV exported from the database and re-imported would preserve only start_time/end_time; all enrichment is lost. This is by design (eval set is intervals-only) but breaks round-trip fidelity.",

"CVAT attribute aliases are LENIENT (ending_reason, outcome, last_shot_outcome (in/net/out) all map to canonical last_shot_outcome), but CSV import/export do NOT tolerate variant column names. If a vendor-submitted CSV has 'outcome' instead of 'last_shot_outcome', it is silently ignored.",

"Frame-based metadata (frame_min, frame_max, n_boxes in CVAT RallyRecord) is NOT preserved in canonical CSV. Once converted to decimal seconds, frame info is lost. RallyRecord.load_canonical_csv sets frame_min/max to -1 as a marker."

```
],
"files": [
  "C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/badminton.py",
  "C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/base.py",
  "C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/pickleball.py",
  "C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/tabletennis.py",
  "C:/Users/avidu/Projects/sports-data-collector/src/core/validator.py",
  "C:/Users/avidu/Projects/sports-data-collector/src/core/parsing.py",
  "C:/Users/avidu/Projects/sports-data-collector/src/core/db.py",
  "C:/Users/avidu/Projects/sports-data-collector/src/parsers/cvat/rally_cvat.py",
  "C:/Users/avidu/Projects/sports-data-collector/src/parsers/badminton/coachai.py",
  "C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py",
  "C:/Users/avidu/Projects/sports-data-collector/docs/annotation/ROORA_BRIEF.md",
  "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/calibrate_wasb.py",
  "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/eval_partial_labels.py",
  "C:/Users/avidu/Projects/rally-annotator/docs/CSV_FORMAT.md",
  "C:/Users/avidu/Projects/rally-annotator/docs/ENDING_REASONS.md"
]
```

```
},
{
  "subsystem": "Sports Data Collector ? CVAT Vendor Ingestion & Pipeline Role",
  "summary": "The collector is the data producer in a two-repo training pipeline. It curates sports videos with per-rally temporal annotations [start, end) and commercial-license metadata, then exports to the badminton-highlight-indexer for processing and offline rally-segmenter training. The Roora vendor ingestion flow converts CVAT XML exports (or canonical CSVs) through a four-command pipeline: convert-cvat (normalizing frame?seconds), validate-delivery (QA gate with blocker checks + human review sheet), import-local (SQLite registration), and export (indexer-ready CSV+manifest). Manual self-rating via the VLC rally-annotator tool provides an alternative, lightweight ingestion path with minimal metadata. Golden-set role: temporal-only annotations for segmentation evaluation and training, explicitly not for frame-level shuttle detection; non-commercial sources excluded from default exports.",
```

```
  "key_facts": [
    "Collector role (PIPELINE_ROLE.md:1-46): data producer feeding badminton-highlight-indexer; goal is offline rally segmenter trained on commercially-licensed, annotated corpus to replace Gemini API",
```

```
    "Roora vendor pipeline (INGESTION_PIPELINE.md:7-44): four sequential commands: convert-cvat ? validate-delivery ? import-local ? export; converts CVAT XML boxes+attributes to canonical CSV with frame?seconds recomputation",
```

```
    "Critical frame-to-time conversion (rally_cvat.py:24-29): recomputes from frame_min/frame_max using true fps; vendor's start_time/end_time attributes explicitly ignored to avoid systematic errors from step-sampled jobs with wrong fps assumptions",
```

```
    "Canonical CSV schema (ROORA_BRIEF.md:14-30): vendor provides rally_number, start_mmss, end_mmss, shots_count, ending_reason, last_shot_outcome, score_before/after, notes; collector auto-fills start_time/end_time as decimal seconds",
```

"Validation rules (INGESTION_PIPELINE.md:68-92): blocker issues (overlaps, zero-duration) gate the import; high/medium issues (shot_density, possible_missed_rally, vocab) populate human review sheet; missed-rally detection via gap heuristic (>15s) and inflated-span flagging",

"Intake manifest fields (INTAKE_MANIFEST_TEMPLATE.csv:1-5): video_id, sport, format, fps (critical), duration_mmss, expected_rallies, assigned_to, status; fps provided to vendor for frame?seconds formula",

"VLC rally-annotator CSV (rally-annotator/README.md:10-14): minimal 5-column format (rally_number, start_time, end_time, ending_reason, sport) with decimal seconds only; no shots_count/scores; resumable (continues numbering on re-open)",

"Rally-annotator ingestion (CSV_FORMAT.md:26-41): direct import via curate import-local; optional fields populated as None; validation still runs (overlaps, positive duration); no external QA gate (self-produced)",

"Quality bar (ROORA_BRIEF.md:79?86): $\pm 0.3s$ boundary tolerance, zero missed rallies, schema compliance, shots_count exact for ?15 shots or ± 1 for longer rallies, 20% human spot-check on every delivery",

"Golden-set temporal-only contract (ADR-002, GOLDEN_SET_IMPLEMENTATION.md:6-8): (video_id, ordinal, start_time, end_time) shape shared across five sports; NOT used for shuttle detector training (lacks per-frame x,y labels); only for temporal segmenter training + eval",

"Regression testing (GS-2, GOLDEN_SET_IMPLEMENTATION.md:51-52): regress(video, tier, tolerance) compares predictions vs GT, flags when precision/recall drop >5% default; baseline snapshots per video with re-snapshot path for legitimate improvements",

"Non-commercial exclusion (INGESTION_PIPELINE.md:124-129): videos kept in DB regardless of license; default export commercial-only; non-commercial sources excluded unless --include-noncommercial flag passed",

"Observability reports (INGESTION_PIPELINE.md:131-144): per-video JSON (machine-readable), technical .md, and customer summary .md emitted on every validate-delivery/convert-cvat/import-local run for debugging/transparency"

],

"contracts": [

"Video identity contract: indexer keys by file MD5; collector by human video_id; manifest carries exact local_path so downstream processing runs on that exact file (acquisition precedes processing)",

"CSV schema contract: collector exports per-video (start, end) CSVs matching indexer's eval format; manifest.json pairs each CSV with video file; sport-agnostic (start_time, end_time) shape across badminton, tennis, cricket, pickleball, tabletennis",

"Canonical CSV format: rally_number (1-based, no gaps), start_time/end_time (decimal seconds), shots_count (optional int), ending_reason (enum: winner/forced_error/unforced_error/service_fault/let/other), last_shot_outcome (enum: in/net/out/n_a), score_before/after (optional 'A-B'), notes (required when ending_reason=other)",

"CVAT attribute aliases: lenient matching for rally metadata (e.g. 'score_before (a-b)' ? 'score_before'); frame-derived times override vendor-supplied times; frame range [frame_min, frame_max] at true fps = [start_time, end_time] seconds",

"Validation blockers: overlaps (end[N] > start[N+1]), zero-duration (end ? start), frame_out_of_range; violations exit non-zero and block import; non-blocker issues populate review sheet for human verification",

"Rally-annotator CSV format: rally_number, start_time (decimal), end_time (decimal), ending_reason, sport; optional columns may be added; consumers read by header name and skip rows where end ? start",

"Commercial license gate: is_commercial flag per video; default export includes only commercial sources; non-commercial explicitly excluded unless --include-noncommercial passed",

"Temporal-only annotation contract: (video_id, ordinal, start_time, end_time) shared across sports; no per-frame (x, y) shuttle coordinates; trained for segmentation (active vs. dead play), not detection; licensed/owned sources only"

],

"gaps_or_mismatches": [

"Partial-label strategy not explicitly documented: collector emphasizes 'zero missed rallies' as non-negotiable but offers no formal framework for handling genuinely ambiguous clips (warm-up detection, occluded play, administrative stoppages marked 'other')",

"Rally-annotator metadata subset: VLC tool omits shots_count, scores, notes ? fields that vendor includes; no documented pathway to enrich rally-annotator CSVs with post-hoc shot counts or scores before/after import",

"Missed-rally detection limitations acknowledged but not solved: gap heuristic (>15s) and shot_density flags are heuristic only; motion-based second-opinion stated as 'future enhancement' with no timeline",

"Quality bar vs. rally-annotator: vendor delivery expects 20% human spot-check; self-produced rally-annotator has no equivalent QA gate (rater is both producer and verifier, single POV)",

"Warm-up/knock-up exclusion rule: golden rule in ANNOTATION_GUIDE.md but no automated detection; rater must visually distinguish real play from pre-match cooperative rallying (relies on human judgment)",

"Frame sampling tolerance: documentation acknowledges ~0.5s sampling slack absorbed by clip padding but doesn't quantify impact on boundary accuracy for edge-case rallies",

"ADR-002 (golden set temporal-only) doesn't address: whether multi-annotator IAA (inter-annotator agreement) will be tracked; single-rater bias not factored into regression tolerance (ADR-004 reuses existing harness, may need sport-specific thresholds)"

```

],
"files": [
  "C:/Users/avidu/Projects/sports-data-collector/docs/PIPELINE_ROLE.md",
  "C:/Users/avidu/Projects/sports-data-collector/docs/INGESTION_PIPELINE.md",
  "C:/Users/avidu/Projects/sports-data-collector/src/parsers/cvat/rally_cvat.py",
  "C:/Users/avidu/Projects/sports-data-collector/docs/annotation/ROORA_BRIEF.md",
  "C:/Users/avidu/Projects/sports-data-collector/docs/annotation/ANNOTATION_GUIDE.md",
  "C:/Users/avidu/Projects/sports-data-collector/docs/annotation/INTAKE_MANIFEST_TEMPLATE.csv",
  "C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py",
  "C:/Users/avidu/Projects/rally-annotator/README.md",
  "C:/Users/avidu/Projects/rally-annotator/docs/CSV_FORMAT.md",
  "C:/Users/avidu/Projects/rally-annotator/docs/ENDING_REASONS.md",
  "C:/Users/avidu/Projects/badminton-highlight-indexer/docs/GOLDEN_SET_IMPLEMENTATION.md",
  "C:/Users/avidu/Projects/badminton-highlight-indexer/docs/archives/decisions/DECISIONS.md"
]
},
{
  "subsystem": "Manual Rally Annotation Artifacts",
  "summary": "Two distinct CSV formats exist for manual rally annotation: (1) the lean rally-annotator output with 5 core columns (rally_number, start_time, end_time, ending_reason, sport), and (2) the richer rally_review format with 14 columns adding metadata for manual curation (shot counts, time-in-minutes-seconds format, verification flags, reviewer comments). The ending_reason taxonomy is shared across both: winner, forced_error, unforced_error, service_fault, let, other. Manual ingesters must handle partial labeling (videos only partially tagged), fractional rally numbers (7.5, 16.5 from corrections), missing end_time values, status flags (VERIFIED/needs review), and free-text notes capturing net touchdowns and flagged anomalies like OVERLAP or INFLATED timing.",
  "key_facts": [
    "rally-annotator CSV columns (5): rally_number (int, 1-based monotonic), start_time (float, decimal seconds), end_time (float, decimal seconds, always > start_time), ending_reason (enum: winner|forced_error|unforced_error|service_fault|let|other), sport (enum: badminton|tennis|table_tennis|pickleball|padel) - CSV_FORMAT.md:6-18",
    "rally_review CSV columns (14): rally (int, rally identifier, may be fractional like 7.5 or 16.5), start_mmss (string, mm:ss.sss format, e.g. '1:02.062'), end_mmss (string, mm:ss.sss format), dur_s (float, duration in seconds), shots (int, shot count for the rally), ending_reason (enum or pipe-delimited alternates like 'forced_error|other'), ending_consistent (yes|NO, verification flag), notes (free-text, e.g. 'net', OVERLAP, INFLATED), auto_flag (status field for automated review), VERIFIED(Y/N) (Y/N or empty), true_start_mmss (optional corrected start), true_end_mmss (optional corrected end), true_ending (optional corrected reason), reviewer_comment (optional notes) - rally_review.csv header",
    "Ending reason taxonomy (6 values, shared across sports): winner (opponent couldn't return in-court shot), forced_error (loser missed under pressure), unforced_error (loser missed routine shot with time+position), service_fault (serve-rule violation), let (rally replayed under rules), other (occluded/administrative/unclear) - ENDING_REASONS.md:7-9",
    "rally_annotator.lua line 88 hard-codes header: 'rally_number,start_time,end_time,ending_reason,sport\\n'",
    "rally_annotator.lua line 262 writes CSV with %.3f precision for floats: 'd,%.3f,%.3f,%s,%s' (e.g. '1,37.434,72.194,winner,badminton')",
    "Actual lean CSV (Adarsh and Avi.rallies.csv): 40 rallies, rally_number 1-40 sequential, start_time range 37.434-599.712s, end_time range 72.194-604.962s, all have winning/error/service_fault labels, all sport=badminton",
    "Actual rich CSV (rally_review.csv): 26 rows, rally column shows integers AND fractional numbers (1,2,3... up to 26 but stored as rally NOT rally_number), times in mm:ss.sss format
  ]
}

```

(e.g. '0:03.003', '2:29.649'), shot counts range 1-15, ending_reason mostly single values but row 16 shows 'forced_error|other' (pipe-delimited alternatives), VERIFIED column has Y, NO, or empty, notes field captures qualitative flags ('net', 'OVERLAP', 'INFLATED')),

"Manual ingester realities: (1) PARTIAL LABELING - video may be only partially tagged, labels cover only a prefix/window of total runtime, (2) FRACTIONAL RALLY NUMBERS - corrections/insertions create numbers like 7.5 or 16.5, not just integers, (3) MISSING end_time - some fields may be blank (not shown in sample but possible), (4) STATUS FLAGS - VERIFIED(Y/N), ending_consistent(yes/NO), auto_flag field mark review state, (5) last_shot_outcome implicit in ending_reason enum, (6) shots_count as distinct column in review format, (7) FREE-TEXT NOTES - unstructured annotations like 'net touchdown' or anomaly flags ('INFLATED timing window', 'OVERLAP with prior rally'))",

"rally_review CSV uses time-in-minutes-seconds string format (mm:ss.sss) while rally-annotator uses decimal-seconds floats (crucial for parsing)",

"rally-annotator v1.3 supports 5 sports: badminton, tennis, table_tennis, pickleball, padel (line 85-87 of Lua)",

"CSV parsing logic in rally_annotator.lua lines 222-250: forgiving parser that preserves extra columns, treats any leading-integer line as a rally row, skips header and blank lines"

],

"contracts": [

"LEAN ANNOTATOR CSV (5 columns): rally_number (int, 1-based, monotonic, continues across reopens), start_time (float, decimal seconds), end_time (float, decimal seconds, > start_time), ending_reason (enum: winner|forced_error|unforced_error|service_fault|let|other), sport (enum: badminton|tennis|table_tennis|pickleball|padel)",

"RICH REVIEW CSV (14 columns): rally (int or float, rally identifier, may be fractional), start_mmss (string, mm:ss.sss format), end_mmss (string, mm:ss.sss format), dur_s (float, seconds), shots (int, shot count), ending_reason (string, may be pipe-delimited alternatives like 'forced_error|other'), ending_consistent (enum: yes|NO), notes (string, free-text), auto_flag (string, status field), VERIFIED(Y/N) (enum: Y|N|empty), true_start_mmss (string or empty), true_end_mmss (string or empty), true_ending (string or empty), reviewer_comment (string or empty)",

"ENDING_REASON TAXONOMY (6 shared values across all sports): winner, forced_error, unforced_error, service_fault, let, other",

"SPORT TAXONOMY (5 supported): badminton, tennis, table_tennis, pickleball, padel",

"Decimal seconds format in lean CSV: floats with arbitrary precision but typically 3 decimal places (millisecond-level). Example: 37.434, 72.194",

"Time format in review CSV: string representation mm:ss.sss (minutes:seconds.milliseconds). Example: '0:03.003', '4:22.762'. Parser must convert to seconds for comparison with annotator output",

"Rally number uniqueness: within one video's CSV, rally_number is unique and monotonic. Fractional numbers (7.5, 16.5) may appear in review CSV when corrections create inserted records"

],

"gaps_or_mismatches": [

"TIME FORMAT MISMATCH: rally-annotator outputs decimal-seconds floats (37.434); rally_review uses mm:ss.sss string format (0:03.003). Ingester must convert review times to decimal-seconds or vice versa for reconciliation",

"COLUMN NAME MISMATCH: lean CSV has 'rally_number' (int counter), but rich CSV uses 'rally' (may be fractional). Manual ingester must handle both naming conventions and account for fractional IDs from edits/corrections",

"MISSING end_time: while both sample CSVs show complete (start, end) pairs, the schema allows blank end_time. Ingester must validate end > start and reject incomplete rallies",

"FRACTIONAL RALLY NUMBERS: rally-annotator logic assumes 1-based integer sequencing (line 283-286 of Lua), but real-world manual edits/insertions in review CSV introduce 7.5, 16.5-style identifiers. Ingester must parse floats, not just integers",

"ENDING_REASON ALTERNATIVES: lean CSV always single enum value; rich CSV row 16 shows 'forced_error|other' (pipe-delimited), indicating human uncertainty or net-touched ambiguity. Ingester must handle both scalar and pipe-delimited alternatives",

"OPTIONAL CORRECTION COLUMNS: rich CSV has true_start_mmss, true_end_mmss, true_ending columns (all empty in sample but semantically present). Ingester must treat these as optional overrides for the original times/reason if populated",

"PARTIAL LABELING: neither sample shows a video labeled only to 50% completion, but ENDING_REASONS.md and Lua code (lines 438-464) reference the pattern. Ingester must accept incomplete, prefix-window labels (e.g. rallies 1-15 out of 50 total)",

"STATUS FLAG SEMANTICS: VERIFIED and ending_consistent use inconsistent enum values (Y/N, yes/NO capitalization varies). Ingester must normalize or handle case-insensitively",


```

"FREE-TEXT NOTES CONTENT: notes field captures sport-specific edge cases ('net touchdown'
in badminton, 'wall bounce' for padel) without schema. Ingestor must preserve as-is or define a
taxonomy if automated handling is needed",
"AUTO_FLAG AND REVIEW STATE: auto_flag field (empty in sample) indicates automated quality
checks triggered (e.g. OVERLAP, INFLATED). Ingestor must define what flags are actionable vs.
informational"
],
"files": [
"C:/Users/avidu/Projects/rally-annotator/docs/CSV_FORMAT.md",
"C:/Users/avidu/Projects/rally-annotator/docs/ENDING_REASONS.md",
"C:/Users/avidu/Projects/rally-annotator/vlc/rally_annotator.lua",
"C:/Users/avidu/Projects/Annotation Setup/Collect/Adarsh and Avi.rallies.csv",
"C:/Users/avidu/Projects/Annotation Setup/rally_review.csv"
]
},
{
"subsystem": "Collector Output Contract for Rally Indexer Eval/Training",
"summary": "The indexer consumes rally ground truth from the collector via a **manifest.json
+ per-video CSV** contract. The manifest (created by `sports-data-collector`'s `curate export`
command) pairs each video with a CSV of decimal-second rally intervals, plus metadata (fps,
resolution, video path, sport, license, curation status). The CSV columns are minimal:
`start,end` in seconds. The indexer's eval and training pipelines use **temporal IoU matching**
(greedy one-to-one at a threshold, default 0.5) to compare predicted rally windows against
ground truth; partial labels are supported only in the dedicated `eval_partial_labels.py`
harness via explicit `--window-start`/`--window-end` flags, not represented in the export format
itself.",
"key_facts": [
"calibrate_wasb.py:34-49 / load_golden_gts() ? reads CSV with columns (case-insensitive):
start_time, end_time (decimal seconds); returns sorted list of (start, end) tuples",
"calibrate_local.py:66-74 / build_videos() ? expects manifest JSON with structure [{name,
trajectory, fps, frame_width, labels}, ...]; labels = path to golden CSV",
"eval_partial_labels.py:47-54 / restrict_to_window() ? implements labeled-window concept:
filter predictions/GT to [window_start, window_end] interval; predicted segments outside window
are dropped (neither rewarded nor penalized)",
"eval_partial_labels.py:68-78 ? window_end defaults to max(end_time) of all GT rallies;
window_start defaults to 0.0; both are CLI args not part of CSV/manifest",
"rally_slicer.py:53-73 / load_rally_labels() ? loads CSV with columns (case-insensitive):
rally_number, start_time, end_time (decimal seconds OR mm:ss/HH:MM:SS); returns dict
rally_number ? (start, end) seconds",
"distill_local.py:81-87 / load_video() ? reads manifest entry {fps, frame_width,
trajectory, labels}; calls load_golden_gts(labels) to get GT intervals; label_candidates()
matches candidate windows to GT via IoU",
"training.py:50-62 / label_candidates() ? uses match_intervals() with iou_threshold
(default 0.5); candidate is positive iff it matches exactly one GT interval at IoU >=
threshold",
"metrics.py:48-79 / match_intervals() ? greedy one-to-one matching: highest-IoU pairs
claimed first, each pred/GT used at most once; only pairs with IoU >= iou_threshold are
considered for matching",
"metrics.py:15-29 / interval_iou() ? temporal intersection-over-union: overlap/(union)
both measured as durations in seconds; returns 0.0 for non-overlapping or degenerate intervals",
"sports-data-collector curate.py:496-509 / export_eval_set() ? manifest entry structure:
{video_id, sport, csv (relative path), local_path, video_url, match_name, fps, resolution,
license_type, is_commercial, status, num_segments}",
"sports-data-collector curate.py:490-494 ? exports CSV with header 'start,end' and rows
[start_seconds, end_seconds] (decimal, computed from DB intervals); one rally per row",
"annotations.py:44-76 / load_annotations() ? legacy simple format: reads CSV with optional
'start'/'start_time' header row; columns 1-2 are start/end timestamps (decimal OR colon-
separated MM:SS/HH:MM:SS); blank lines and # comments ignored",
"calibrate_local.py:19-20 ? manifest expected from collector at: `../sports-data-
collector/data/eval_export/manifest.json` (relative path hint in docs)",
"batch.py:22-40 / resolve_video_path() ? normalizes manifest local_path: strips './',
converts backslashes to forward slashes, joins with videos_root if relative; cross-platform
handling for Windows drive letters (C:/...)",
"eval_partial_labels.py:72-82 ? labeled-window enforcement: load_golden_gts() gets all
rallies; manually compute/override window_end from CLI; restrict_to_window() filters both

```

```

predictions and GT to in-window region before scoring",
    "distill_local.py:48-69 / build_candidates() ? proposal stage (velocity_thresh=0.005,
merge_gap=1.0, min_dur=0.5) generates many candidates; WASB trajectory parses trajectory.csv
with columns Frame, Visibility, X, Y"
],
"contracts": [
    "CSV (ground truth): columns start,end | format: decimal seconds (float) | one rally per
row | optional header 'start' or 'start_time' | blank/# lines ignored | no other columns
required/consumed",
    "CSV (human annotations, collector intake): columns rally_number, start_time, end_time,
ending_reason, sport [optional: start_mmss, end_mmss, shots_count, score_before, score_after,
last_shot_outcome, notes] | start/end can be decimal seconds OR MM:SS / HH:MM:SS | rally_number
= string (supports inserted IDs like '7.5') | indexer ignores all columns except
start_time/end_time",
    "manifest.json structure: {commercial_only, statuses, total_videos, total_segments,
eval_sets: [{video_id, sport, csv (relative path to start,end CSV), local_path, video_url,
match_name, fps (float), resolution (string or null), license_type, is_commercial (bool),
status, num_segments}, ...]}",
    "Trajectory CSV (WASB input): columns Frame, Visibility, X, Y | parsed by
TrackNetRunner.parse_trajectory_csv() | motion data required only for calibration/distillation,
not for evaluating labels alone",
    "IoU matching contract: temporal intervals (start, end) in seconds | IoU >= threshold ?
match candidate | greedy one-to-one ? TP counted | threshold defaults to 0.5, configurable via
--iou flag",
    "Labeled-window concept (partial labels): window_start, window_end (seconds) | predicted
intervals outside [window_start, window_end) are dropped before scoring | GT intervals assumed
already within window | NOT part of manifest/CSV; applied only in eval_partial_labels.py via CLI
args",
    "Metadata contract (in manifest): fps (float, required for trajectory windowing and
feature extraction) | frame_width (float, required for WASB velocity normalization) | local_path
(required, string, file system path; used as key with file MD5 for cross-repo identification) |
resolution (optional, string like '1080p')",
    "Feature extraction for training: candidate window bounds ? {duration, peak_velocity,
mean_velocity, active_frame_count (YOLO) OR active_ratio (WASB), track_id_count (YOLO) OR
net_crossings (WASB)} | features must be fps/resolution-INVARIANT except where explicitly
normalized by frame_width"
],
"gaps_or_mismatches": [
    "Labeled-window restriction (eval_partial_labels.py concept) is NOT representable in the
current collector export format ? it is enforced only as CLI args to eval_partial_labels.py, not
persisted in manifest/CSV. If collector needs to support partial labels at scale, a new optional
manifest field (e.g., window_start, window_end) would be required.",
    "Rally metadata (ending_reason, last_shot_outcome, score_before/after, shots_count) are
collected and validated by the collector but NOT consumed by indexer eval/training pipelines ?
they are discarded after validation. Only (start_time, end_time) flow downstream.",
    "Trajectory CSV path is NOT in the manifest ? it is supplied separately as a CLI arg
(--trajectory) to calibrate_wasb, calibrate_local, eval_partial_labels, and distill_local. The
distill pipeline requires it to build features but there is no contract guaranteeing it exists
or is co-located.",
    "Resolution field in manifest is optional (nullable) and rarely used downstream ? fps is
required but resolution is mainly for observability. Feature extraction normalizes by
frame_width instead of resolution string.",
    "Partial-label support assumes GT labels cover a PREFIX of the video only (every rally up
to window_end is labeled, everything after is unlabeled). This assumption is NOT validated or
documented in the manifest ? it is left to the user to enforce via --window-end.",
    "CSV header detection (start, start_time) is case-insensitive but relies on exact column
name matching (no fuzzy matching) ? a vendor sending 'start_seconds' or 'start_frame' would fail
silently or cause the harness to read the wrong column.",
    "No contract for multi-sport data in a single manifest ? eval_sets can contain mixed
sports but most eval/training scripts assume homogeneous sport (e.g., training.label_candidates
does not filter by sport, so mixing sports in one training set would train a sport-agnostic
model).",
    "IoU threshold is a global setting (--iou flag) applied uniformly across all videos in a
manifest ? no per-video or per-sport threshold override supported in the current contract."
],

```

```

"files": [
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\calibrate_wasb.py",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\calibrate_local.py",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\eval_partial_labels.py",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\tools\\rally_slicer.py",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\distill_local.py",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\training.py",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\metrics.py",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\annotations.py",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\batch.py",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\run_phase3_eval.py",
    "C:\\Users\\avidu\\Projects\\sports-data-collector\\src\\cli\\curate.py"
],
{
    "subsystem": "Sports Data Collector - DB/Provenance & Observability Layer",
    "summary": "The collector stores video metadata and sport-specific annotations (rallies, points, deliveries) in SQLite with two-level schema: a central `videos` table holding provenance (source URL, license, sport, video_id) and curation status (STAGING/CURATED/REJECTED), plus sport-specific child tables (badminton_rallies, tennis_points, cricket_deliveries, etc.) containing segment metadata. Observability is layered via obsreport (soft dependency) which generates per-video reports at import/validation stages, capturing parse/validation/registration metrics and flags for delivery issues; local imports bypass vendor-issued flags but still emit reports showing license, commerciality, and segment counts at registration.",
    "key_facts": [
        "C:/Users/avidu/Projects/sports-data-collector/src/core/db.py:52-71: videos table schema ? video_id (PK), sport, license_type (enum: MIT/Apache/BSD/CC0/CC-BY/CC-BY-NC/CC-BY-NC-SA/Public-Domain/Proprietary/Unknown), license_url, is_commercial (boolean flag), status (enum: staging/curated/rejected), match_name, fps, resolution; foreign keys ensure on-delete cascade to child tables.",
        "C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/base.py:24-27: CurationStatus enum ? STAGING, CURATED, REJECTED; used for video-level workflow state only, no segment-level maturity.",
        "C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/base.py:12-22: LicenseType enum ? includes Proprietary (self-rated), MIT/Apache/BSD/CC0/CC-BY (vendor-delivered), CC-BY-NC/CC-BY-NC-SA (non-commercial), PUBLIC_DOMAIN, UNKNOWN; no source tracking (self-rated vs vendor-delivered not distinguished).",
        "C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py:109-122: import_local sets all local imports to CURATED status by default (line 117: 'Custom local imports bypass staging by default'), skipping vendor staging workflow.",
        "C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py:73-78: License commercial-safety check via validator.check_license_commercial(license_type) ? whitelist includes Proprietary (line 21 in validator.py), so self-rated Proprietary is treated as commercial-safe.",
        "C:/Users/avidu/Projects/sports-data-collector/src/reporting/adapters.py:113-156: build_import_recorder() ? observability envelope for import-local; parse stage logs records_parsed; validation stage reports is_valid (boolean); registration stage captures sport, license (value string), is_commercial, status (value string); no annotation author/timestamp/maturity.",
        "C:/Users/avidu/Projects/sports-data-collector/src/reporting/spec.yaml:25-47: Spec rules emit two flags at import: license_noncommercial (warn if non-commercial), missing_fps (warn if fps null). Rules only check video-level fields; no segment-level flags for imports.",
        "C:/Users/avidu/Projects/sports-data-collector/src/core/db.py:210-237: insert_badminton_rallies() ? clears existing rallies for video_id then bulk-inserts new ones; no versioning, audit trail, or per-segment metadata beyond sport schema (rally_number, start_time, end_time, score, shots_count, ending_reason, last_shot_outcome).",
        "C:/Users/avidu/Projects/sports-data-collector/src/reporting/__init__.py:48-77: emit_import_report() ? called from curate.py:145-154 after DB insert; optional (soft dependency); writes next to video or to fallback_dir if video absent; always succeeds (never raises).",
    ]
}

```

```

"C:/Users/avidu/Projects/sports-data-collector/src/parsers/cvat/rally_cvat.py: Issue class
? severity (blocker/high/medium/low), code, message, optional rally_number; only delivery
validation emits Issue objects; import-local does not detect or report Issues.",
"C:/Users/avidu/Projects/sports-data-collector/src/core/validator.py:29-144:
validate_*_rallies() ? checks temporal non-overlap and positive duration; no quality metrics,
annotator tracking, or maturity scores.",
"C:/Users/avidu/Projects/sports-data-collector/tests/test_reporting.py:83-112: import
observability envelope captures only is_valid (boolean), errors (list of strings), count
(segments), sport, source_format; no flags for import-local validation issues."
],
"contracts": [
  "VideoMetadata schema: video_id, sport(enum), video_url?, local_path?, license_type(enum),
  license_url?, is_commercial(bool), status(CurationStatus enum: staging/curated/rejected),
  match_name, fps?, resolution?",
  "CurationStatus enum: STAGING | CURATED | REJECTED ? video-level only, no segment maturity
  states",
  "LicenseType enum: MIT, Apache-2.0, BSD, CC0, CC-BY, CC-BY-NC, CC-BY-NC-SA, Public-Domain,
  Proprietary, Unknown ? no source marker (self-rated vs vendor-delivered)",
  "is_commercial: boolean derived from license_type whitelist check; whitelist in
  config.yaml includes Proprietary (defaults to [MIT, Apache-2.0, BSD, CC0, CC-BY, Public-Domain,
  Proprietary])",
  "Import observability envelope (obsreport.RunRecorder): stages=(parse, validation,
  registration); parse.metrics.records_parsed; validation.status(ok/error),
  validation.metrics.valid(bool); registration.metrics.segments_registered(int),
  registration.details={sport, license(str), is_commercial(bool), status(str)}",
  "Delivery observability envelope: stages=(parse, validation);
  parse.metrics.rallies_parsed; validation.metrics.issues_blocker/high/medium/low; flags carry
  code, severity(error/warn/info), message, stage, customer_message?, evidence={rally_number?,
  rally_frames?, fps?}",
  "Spec rules: license_noncommercial (warn if is_commercial==false), missing_fps (warn if
  fps==null); rules trigger only at import/delivery validation, never at re-tagging",
  "RallyRecord (CVAT adapter): rally_number, start_time, end_time, shots_count?,
  ending_reason?, last_shot_outcome?, score_before?, score_after?, notes?, frame_min, frame_max,
  n_boxes, vendor_start_time?, vendor_end_time?",
  "Database cascade: video DELETE triggers cascade on child tables (badminton_rallies,
  tennis_points, cricket_deliveries, pickleball_rallies, tabletennis_rallies); child tables have
  no independent delete cascade"
],
"gaps_or_mismatches": [
  "No annotation source tracking: LicenseType enum includes Proprietary but does not
  distinguish self-rated (manual local import) from vendor-delivered (CVAT/external parser);
  license provenance assumes external source unless registered via import-local, but no explicit
  marker.",
  "No segment-level maturity: segments (rallies/points/deliveries) lack status/maturity
  fields; video-level STAGING/CURATED/REJECTED does not map to individual segment review state
  (needs_review -> confirmed -> gold not trackable).",
  "No annotator/author metadata: neither VideoMetadata nor rally schemas capture who
  created/reviewed annotations; no timestamp for import or re-tagging.",
  "Local import skips staging workflow: import_local (line 117 in curate.py) hardcodes
  CURATED status, bypassing vendor review pipeline; observability report does not flag this
  deviation.",
  "Import validation does not emit observability flags:
  DatasetValidator.validate_*_rallies() only check temporal integrity (overlaps, duration);
  emit_import_report() logs errors only as string list, not structured Flags; no per-segment QA
  metrics.",
  "No re-tagging audit trail: insert_badminton_rallies() (line 215-216 in db.py) DELETE-
  then-INSERT erases history when re-importing annotations for the same video_id; no versioning or
  conflict detection.",
  "No label quality regression tracking: no metrics for comparing old vs. new annotations
  when re-tagging (e.g., shot_count changes, ending_reason divergence); no drift detection.",
  "License whitelist hardcoded in config fallback: check_license_commercial()
  (validator.py:21) defaults to fixed list including Proprietary; no per-video or per-sport
  override; no rationale/audit trail for why Proprietary is commercial-safe.",
  "Observability rules only video-level: spec.yaml:25-47 defines only video-scope rules
  (license_noncommercial, missing_fps); no rules for segment defects at import time (gaps,

```

```

outliers, schema violations).",
    "No import-local annotation Issues: CVAT delivery parser emits Issue objects
(blocker/high/medium/low) but import-local does not detect or report Issues to observability
envelope; local vs. vendor validation asymmetry."
],
"files": [
    "C:/Users/avidu/Projects/sports-data-collector/src/core/db.py",
    "C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/base.py",
    "C:/Users/avidu/Projects/sports-data-collector/src/core/validator.py",
    "C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py",
    "C:/Users/avidu/Projects/sports-data-collector/src/cli/ingest.py",
    "C:/Users/avidu/Projects/sports-data-collector/src/reporting/__init__.py",
    "C:/Users/avidu/Projects/sports-data-collector/src/reporting/adapters.py",
    "C:/Users/avidu/Projects/sports-data-collector/src/reporting/spec.yaml",
    "C:/Users/avidu/Projects/sports-data-collector/src/parsers/cvat/rally_cvat.py",
    "C:/Users/avidu/Projects/sports-data-collector/tests/test_reporting.py",
    "C:/Users/avidu/Projects/sports-data-collector/tests/test_validator.py",
    "C:/Users/avidu/Projects/sports-data-collector/tests/test_local_import.py"
]
}
]

```

2. assistant (2026-06-08T08:43:43.524Z)

I'll ground my analysis in the findings, but let me verify a few load-bearing facts in the actual code before proposing the canonical-hub architecture.

3. user (2026-06-08T08:43:44.902Z)

```

52     def import_local(self, args):
53         """Manually registers a local video and annotation file into the database."""
54         sport_str = args.sport.lower()
55         if sport_str not in self.config.get("supported_sports", ["badminton"]):
56             print(f"Error: Sport '{sport_str}' is not supported. Supported:
{self.config.get('supported_sports')}")
57             sys.exit(1)
58
59         # Validate video path
60         if not os.path.exists(args.video_path):
61             print(f"Warning: Local video path '{args.video_path}' does not exist on
disk, but continuing registration.")
62
63         # Validate annotation file
64         if not os.path.exists(args.annotations_path):
65             print(f"Error: Annotations file '{args.annotations_path}' not found.")
66             sys.exit(1)
67
68         match_name = args.match_name or os.path.basename(args.video_path).split('.')[0]
69         video_id = args.video_id or match_name.lower().replace(" ", "_")
70
71         # Parse license
72         try:
73             license_type = LicenseType(args.license)
74         except ValueError:
75             print(f"Warning: Unknown license '{args.license}'. Defaulting to Unknown
(non-commercial).")
76             license_type = LicenseType.UNKNOWN
77
78         is_commercial = self.validator.check_license_commercial(license_type)
79
80         # 1. Parse Annotations
81         annotations = []
82         try:
83             annotations = self._parse_local_annotations(sport_str, video_id,
args.annotations_path)

```

```

84         except Exception as e:
85             print(f"Error parsing annotations: {e}")
86             sys.exit(1)
87
88     # 2. Run Validation
89     is_valid = True
90     errors = []
91     if sport_str == "badminton":
92         is_valid, errors = self.validator.validate_badminton_rallies(annotations)
93     elif sport_str == "tennis":
94         is_valid, errors = self.validator.validate_tennis_points(annotations)
95     elif sport_str == "cricket":
96         is_valid, errors = self.validator.validate_cricket_deliveries(annotations)
97     elif sport_str == "pickleball":
98         is_valid, errors = self.validator.validate_pickleball_rallies(annotations)
99     elif sport_str == "tabletennis":
100         is_valid, errors = self.validator.validate_tabletennis_rallies(annotations)
101
102     if not is_valid:
103         print(f"Error: Annotations failed validation!")
104         for err in errors:
105             print(f" - {err}")
106         sys.exit(1)
107
108     # 3. Create Video Metadata
109     video_meta = VideoMetadata(
110         video_id=video_id,
111         sport=SportType(sport_str),
112         video_url=None,
113         local_path=args.video_path,
114         license_type=license_type,
115         license_url=None,
116         is_commercial=is_commercial,
117         status=CurationStatus.CURATED, # Custom local imports bypass staging by
default
118         match_name=match_name,
119         fps=args.fps,
120         resolution=args.resolution
121     )
122
123     # 4. Insert into Database
124     self.db.insert_video(video_meta)
125
126     count = 0
127     if sport_str == "badminton":
128         count = self.db.insert_badminton_rallies(annotations)
129     elif sport_str == "tennis":
130         count = self.db.insert_tennis_points(annotations)
131     elif sport_str == "cricket":
132         count = self.db.insert_cricket_deliveries(annotations)
133     elif sport_str == "pickleball":
134         count = self.db.insert_pickleball_rallies(annotations)
135     elif sport_str == "tabletennis":
136         count = self.db.insert_tabletennis_rallies(annotations)
137
138     print(f"\nSuccessfully imported local video '{match_name}':")
139     print(f" - Video ID: {video_id}")
140     print(f" - Sport: {sport_str}")
141     print(f" - Commercial Use Allowed: {is_commercial} (License:
{license_type.value})")
142     print(f" - Registered {count} annotated segments.\n")
143
144     # Per-video observability report (next to the video). Never raises.
145     from src.reporting import emit_import_report
146     src_fmt = os.path.splitext(args.annotations_path)[1].lstrip(".").lower() or

```

```

147         report_paths = emit_import_report(
148             video_meta_obj=video_meta, annotations=annotations, is_valid=True,
149             errors=[], count=count, sport=sport_str, source_format=src_fmt,
150             fallback_dir=os.path.dirname(os.path.abspath(args.annotations_path)),
151         )
152     if report_paths:
153         print(f" - Observability report: {report_paths.get('technical_md')}")
154         print(f" - Customer summary      : {report_paths.get('customer_md')}\n")
155
156 def _parse_local_annotations(self, sport: str, video_id: str, filepath: str) ->
List[Any]:
157     """Parses simple JSON or CSV annotations files."""
158     parsed_items = []
159     ext = os.path.splitext(filepath)[1].lower()
160
161     if ext == '.json':
162         with open(filepath, 'r', encoding='utf-8') as f:
163             data = json.load(f)
164
165         if not isinstance(data, list):
166             raise ValueError("JSON file must contain a list of objects.")
167
168         # Uses the same safe casts as the CSV path: a missing/blank required
169         # time raises a clear ValueError (caught by the caller) instead of a
170         # bare KeyError, and optional ints degrade to sensible defaults.
171         for idx, obj in enumerate(data):
172             if sport == "badminton":
173                 parsed_items.append(BadmintonRally(
174                     video_id=video_id,
175                     rally_number=_safe_int(obj.get("rally_number"), idx + 1),
176                     start_time=_safe_float(obj.get("start_time")),
177                     end_time=_safe_float(obj.get("end_time")),
178                     score_before=obj.get("score_before"),
179                     score_after=obj.get("score_after"),
180                     shots_count=_safe_int(obj.get("shots_count"), None),
181                     ending_reason=obj.get("ending_reason"),
182                     last_shot_outcome=obj.get("last_shot_outcome")
183                 ))
184             elif sport == "tennis":
185                 parsed_items.append(TennisPoint(
186                     video_id=video_id,
187                     point_number=_safe_int(obj.get("point_number"), idx + 1),
188                     start_time=_safe_float(obj.get("start_time")),
189                     end_time=_safe_float(obj.get("end_time")),
190                     set_score=obj.get("set_score"),
191                     game_score=obj.get("game_score"),
192                     server=obj.get("server"),
193                     ending_type=obj.get("ending_type")
194                 ))
195             elif sport == "cricket":
196                 parsed_items.append(CricketDelivery(
197                     video_id=video_id,
198                     over=_safe_int(obj.get("over"), 0),
199                     ball=_safe_int(obj.get("ball"), 0),
200                     start_time=_safe_float(obj.get("start_time")),
201                     end_time=_safe_float(obj.get("end_time")),
202                     batsman=obj.get("batsman"),
203                     bowler=obj.get("bowler"),
204                     runs=_safe_int(obj.get("runs"), None),
205                     wicket=bool(obj.get("wicket", False)),
206                     extras=_safe_int(obj.get("extras"), 0)
207                 ))
208             elif sport == "pickleball":
209                 parsed_items.append(PickleballRally(

```

```

210             video_id=video_id,
211             rally_number=_safe_int(obj.get("rally_number"), idx + 1),

```

4. user (2026-06-08T08:43:45.475Z)

```

356 def write_canonical_csv(records: List[RallyRecord], out_path: str) -> None:
357     """Write records as the canonical per-rally CSV that ``import-local`` reads."""
358     import csv
359     os.makedirs(os.path.dirname(os.path.abspath(out_path)), exist_ok=True)
360     with open(out_path, "w", newline="", encoding="utf-8") as f:
361         w = csv.writer(f)
362         w.writerow(CANONICAL_COLUMNS)
363         for r in sorted(records, key=lambda x: x.rally_number):
364             w.writerow([
365                 r.rally_number,
366                 _mss(r.start_time), _mss(r.end_time),
367                 f"{r.start_time:.3f}", f"{r.end_time:.3f}",
368                 r.shots_count if r.shots_count is not None else "",
369                 r.ending_reason or "",
370                 r.last_shot_outcome or "",
371                 r.score_before or "", r.score_after or "", r.notes or "",
372             ])
373
374
375 def render_issue_report(result: ConversionResult, source: str) -> str:
376     """A human-readable issue report (for review / sending back to the vendor)."""
377     lines = [
378         f"CVAT rally conversion report ? {source}",
379         f"  fps used          : {result.fps:.3f}",
380         f"  sampling step      : {result.step} frames (~{result.step / result.fps:.2f}s)",
381         f"  rallies parsed     : {len(result.records)}",
382         f"  issues              : {len(result.issues)} ({result.n_blockers} blocker)",
383         "",
384     ]
385     if not result.issues:
386         lines.append("No issues detected.")
387     else:
388         for sev in ("blocker", "high", "medium", "low"):
389             group = [i for i in result.issues if i.severity == sev]
390             if not group:
391                 continue
392             lines.append(f"[{sev.upper()}]")
393             for i in group:
394                 lines.append(f"  - ({i.code}) {i.message}")
395             lines.append("")
396     return "\n".join(lines)
397
398
399 # ----- #
400 # Validation reporting ? used by `curate validate-delivery`. #
401 # These turn a (records, issues) pair into the artifacts a human reviews: #
402 # a machine summary (JSON), a human report (md), and an annotatable sheet. #
403 # ----- #
404
405 def _parse_secs(value: Optional[str]) -> Optional[float]:
406     """Decimal seconds or mm:ss / hh:mm:ss -> float; None if blank/unparseable."""
407     s = (value or "").strip()
408     if not s:
409         return None
410     if ":" in s:
411         try:
412             total = 0.0
413             for part in s.split(":"):
414                 total = total * 60 + float(part)
415             return total

```



```

416         except ValueError:
417             return None
418     try:
419         return float(s)
420     except ValueError:
421         return None
422
423
424 def load_canonical_csv(csv_path: str) -> List[RallyRecord]:
425     """Load a canonical per-rally CSV into RallyRecords, so a *CSV* delivery (not
426     just a CVAT export) can be validated. Decimal start/end preferred, mm:ss
427     fallback; rows with no usable start/end are skipped. (frame_* are -1 here, as a
428     CSV carries no frame info ? only the frame-derived checks become inapplicable.)
429     """
430     import csv as _csv
431     if not os.path.exists(csv_path):
432         raise FileNotFoundError(f"CSV not found: {csv_path}")
433     records: List[RallyRecord] = []
434     with open(csv_path, newline="", encoding="utf-8") as f:
435         reader = _csv.DictReader(f)
436         reader.fieldnames = [(n or "").strip().lower() for n in (reader.fieldnames or
437         [])]
438         for raw in reader:
439             row = {(k or "").strip().lower(): v for k, v in raw.items()}
440             rn = safe_int(row.get("rally_number"), None)
441             if rn is None:
442                 continue
443             start = _parse_secs(row.get("start_time"))
444             if start is None:
445                 start = _parse_secs(row.get("start_mmss"))
446             end = _parse_secs(row.get("end_time"))
447             if end is None:
448                 end = _parse_secs(row.get("end_mmss"))
449             if start is None or end is None:
450                 continue
451             records.append(RallyRecord(
452                 rally_number=rn, start_time=start, end_time=end,
453                 shots_count=safe_int(row.get("shots_count"), None),
454                 ending_reason=(row.get("ending_reason") or "").strip() or None,
455                 last_shot_outcome=(row.get("last_shot_outcome") or "").strip() or None,
456                 score_before=(row.get("score_before") or "").strip() or None,
457                 score_after=(row.get("score_after") or "").strip() or None,
458                 notes=(row.get("notes") or "").strip() or None,
459                 frame_min=-1, frame_max=-1, n_boxes=0,
460             ))
461         return records
462
463 def build_summary(records: List[RallyRecord], issues: List[Issue],
464                 source: str = "", fps: Optional[float] = None) -> Dict[str, Any]:
465     """Machine-readable validation summary (JSON / CI)."""

```

5. assistant (2026-06-08T08:43:47.958Z)

Now let me verify the canonical columns constant and the export path to confirm the asymmetry between rich import and intervals-only export.

6. user (2026-06-08T08:43:49.398Z)

```

40     from collections import Counter
41     from dataclasses import dataclass, field
42     from typing import Any, Dict, List, Optional, Tuple
43
44     from src.core.parsing import safe_int
45     from src.core.downloader import probe_resolution_fps

```

```

46
47 # The agreed cross-sport ending_reason vocabulary for net-separated sports.
48 # Kept here as a soft check only (a warning, never a hard failure) so this
49 # adapter does not depend on the annotation-spec wording.
50 DEFAULT_ALLOWED_REASONS = {
51     "winner",
52     "forced_error",
53     "unforced_error",
54     "service_fault",
55     "let",
56     "other",
57 }
58
59 # The observational outcome of the last shot ? orthogonal to ending_reason
60 # (winner can be 'in'; an error can be 'net' or 'out'; service_fault/let -> 'n_a').
61 DEFAULT_ALLOWED_OUTCOMES = {"in", "net", "out", "n_a"}
62
63 # CVAT attribute names we map onto canonical columns. The vendor's CVAT label
64 # schema has used suffixed names like "score_before (A-B)"; we match leniently.
65 _ATTR_ALIASES = {
66     "rally_number": ("rally_number",),
67     "shots_count": ("shots_count",),
68     "ending_reason": ("ending_reason",),
69     "last_shot_outcome": ("last_shot_outcome", "outcome", "last_shot_outcome
(in/net/out)"),
70     "score_before": ("score_before", "score_before (a-b)"),
71     "score_after": ("score_after", "score_after (a-b)"),
72     "notes": ("notes", "note"),
73 }
74
75 CANONICAL_COLUMNS = [
76     "rally_number", "start_mmss", "end_mmss", "start_time", "end_time",
77     "shots_count", "ending_reason", "last_shot_outcome", "score_before", "score_after",
"notes",
78 ]
79
80
81 @dataclass
82 class RallyRecord:
83     """One rally recovered from the CVAT export, time re-derived from frames."""
84     rally_number: int
85     start_time: float          # seconds, = frame_min / fps
86     end_time: float           # seconds, = frame_max / fps (see padding note)
87     shots_count: Optional[int]
88     ending_reason: Optional[str]
89     last_shot_outcome: Optional[str]
90     score_before: Optional[str]
91     score_after: Optional[str]
92     notes: Optional[str]
93     frame_min: int
94     frame_max: int
95     n_boxes: int
96     vendor_start_time: Optional[str] = None    # the (discarded) attribute value
97     vendor_end_time: Optional[str] = None
98
99     @property
100     def duration(self) -> float:
101         return self.end_time - self.start_time
102
103
104 @dataclass
105 class Issue:
106     """A defect found in the delivery. ``blocker`` issues stop a clean import."""
107     severity: str    # 'blocker' | 'high' | 'medium' | 'low'
108     code: str

```



```

496         manifest.append({
497             "video_id": video.video_id,
498             "sport": sport.value,
499             "csv": os.path.relpath(csv_path, out_dir).replace(os.sep, "/"),
500             "local_path": video.local_path,
501             "video_url": video.video_url,
502             "match_name": video.match_name,
503             "fps": video.fps,
504             "resolution": video.resolution,
505             "license_type": video.license_type.value,
506             "is_commercial": video.is_commercial,
507             "status": video.status.value,
508             "num_segments": len(intervals),
509         })
510
511     os.makedirs(out_dir, exist_ok=True)
512     manifest_doc = {
513         "commercial_only": commercial_only,
514         "statuses": [s.value for s in statuses],
515         "total_videos": len(manifest),
516         "total_segments": sum(m["num_segments"] for m in manifest),
517         "eval_sets": manifest,
518     }
519     manifest_path = os.path.join(out_dir, "manifest.json")
520     with open(manifest_path, "w", encoding="utf-8") as f:
521         json.dump(manifest_doc, f, indent=2)
522
523     print(f"\n=== Exported eval/training set to '{out_dir}' ===")
524     if manifest:
525         summary = [[m["sport"], m["video_id"], m["num_segments"],
526                     "YES" if m["is_commercial"] else "NO",
527                     os.path.basename(m["local_path"]) if m["local_path"] else "(no
local file)"]
528                     for m in manifest]
529         print(tabulate(summary, headers=["Sport", "Video ID", "Segments", "Comm?",
"Video file"], tablefmt="grid"))
530         print(f"\nVideos exported : {len(manifest)}")
531         print(f"Total segments : {manifest_doc['total_segments']}")
532         if skipped_noncomm:
533             print(f"Skipped (non-commercial) : {skipped_noncomm} (pass --include-
noncommercial to include)")
534         if skipped_empty:
535             print(f"Skipped (no annotations) : {skipped_empty}")
536         print(f"Manifest : {manifest_path}\n")
537
538     def convert_cvat(self, args):
539         """Convert a vendor CVAT rally export into our canonical per-rally CSV.

```

8. assistant (2026-06-08T08:43:54.378Z)

The findings are confirmed by the code. The architecture is already mostly in the canonical-hub shape ? collector is the producer, indexer consumes manifest+CSV ? but the three ingestion paths (manual VLC, vendor CVAT, vendor CSV) enter through asymmetric doors, and the export is intervals-only. Let me confirm the validator's overlap/duration checks since several recommendations key off them.

9. user (2026-06-08T08:43:55.153Z)

```

1     from typing import List, Tuple, Union
2     from src.core.config import load_config
3     from src.core.schemas.base import VideoMetadata, LicenseType
4     from src.core.schemas.badminton import BadmintonRally
5     from src.core.schemas.tennis import TennisPoint
6     from src.core.schemas.cricket import CricketDelivery
7     from src.core.schemas.pickleball import PickleballRally

```

```

8     from src.core.schemas.tabletennis import TableTennisRally
9
10    class DatasetValidator:
11        def __init__(self, config_path: str = "config/config.yaml"):
12            self.config_path = config_path
13            self.commercial_licenses = self._load_licenses_whitelist()
14
15        def _load_licenses_whitelist(self) -> List[str]:
16            # load_config returns {} on a missing/unreadable file, so the default list
17            # below is the single fallback (keep in sync with config.yaml).
18            config = load_config(self.config_path)
19            return config.get(
20                'commercial_licenses',
21                ["MIT", "Apache-2.0", "BSD", "CC0", "CC-BY", "Public-Domain",
22                "Proprietary"],
23            )
24
25        def check_license_commercial(self, license_type: Union[LicenseType, str]) -> bool:
26            """Determines if a license type is safe for commercial use based on the
27            configuration whitelist."""
28            val = license_type.value if isinstance(license_type, LicenseType) else
29            license_type
30            return val in self.commercial_licenses
31
32        def validate_badminton_rallies(self, rallies: List[BadmintonRally]) -> Tuple[bool,
33        List[str]]:
34            """
35            Validates badminton rallies:
36            - Sorted chronologically.
37            - Start time < End time (handled by Pydantic, but double checked).
38            - No temporal overlap between sequential rallies.
39            """
40            errors = []
41            if not rallies:
42                return True, errors
43
44            # Sort by rally number to check progression
45            sorted_rallies = sorted(rallies, key=lambda r: r.rally_number)
46
47            for i in range(len(sorted_rallies)):
48                r = sorted_rallies[i]
49                if r.start_time >= r.end_time:
50                    errors.append(f"Rally {r.rally_number}: start_time ({r.start_time}) >=
51                    end_time ({r.end_time})")
52
53                if i > 0:
54                    prev = sorted_rallies[i - 1]
55                    if r.start_time < prev.end_time:
56                        errors.append(
57                            f"Rally overlap detected between Rally {prev.rally_number} "
58                            f"(ends at {prev.end_time}s) and Rally {r.rally_number} "
59                            f"(starts at {r.start_time}s)"
60                        )
61
62            return len(errors) == 0, errors
63
64        def validate_tennis_points(self, points: List[TennisPoint]) -> Tuple[bool,
65        List[str]]:
66            errors = []

```

10. assistant (2026-06-08T08:43:57.515Z)

All findings are confirmed against the code. I have what I need to deliver the canonical-hub architecture proposal.

11. user (2026-06-08T08:45:20.032Z)

Structured output provided successfully

12. assistant (2026-06-08T08:45:31.297Z)

I've delivered the canonical-hub architecture proposal via StructuredOutput. Summary of what I grounded and recommended:

****Core thesis:**** The collector is already 80% a canonical hub ? it's the sole producer and the indexer consumes only `manifest.json` + `start,end` CSVs (verified at `curate.py:440-536` and `calibrate_wasb.py:34-49`). The gap is that the three sources enter through asymmetric doors: vendor CVAT gets the rich adapter with Issue detection (`rally_cvat.py:325-460`), while manual VLC/review CSV and vendor CSV funnel through a thinner parser (`curate.py:156-302`) that reads fewer fields, tolerates no aliases, and emits no Issues.

****The six recommendations, in priority order:****

- ****P0**** ? Unify all sources behind one `RallyAdapter` protocol emitting `RallyRecord` + `List[Issue]`, with one internal normal form.
- ****P0**** ? Make the manual-CSV adapter actually tolerate the real `rally\_review.csv` shape (mm:ss, fractional ids, pipe-delimited reasons, `true\_` corrections) by reusing the existing `\_parse\_secs`/`load\_canonical\_csv` helpers.
- ****P1**** ? Run Issue detection on every source, fixing the inversion where self-rated manual data gets *less* scrutiny than vendor deliveries.
- ****P1**** ? Record `annotation\_source` provenance and stop auto-CURATING manual imports (route to STAGING).
- ****P1**** ? Make the labeled window a first-class manifest field so partial labels stop poisoning precision.
- ****P2**** ? Freeze and document the intervals-only export as the single narrow seam.

Every claim is cited to a verified file/line, and I confirmed the load-bearing facts (the thin vs. rich parser split, the intervals-only export at `curate.py:490-494`, the `CANONICAL\_COLUMNS` constant, and the shared validator) by reading the actual code rather than relying solely on the findings.